

MTB Personal Loan Credit Scorecard

A Plain-English Guide to the Project
and How to Run It, Step by Step

Meridian Trust Bank — Personal Loan Application Scorecard

Companion guide to the full technical project
Covers: what this project is, why each part exists,
and exactly which file to run, in what order

Part 1 — What This Project Actually Is

Explained without jargon, from the ground up

The one-sentence version

This project builds a system that looks at a person applying for a personal loan and predicts, using their financial history, **how likely they are to fail to repay it** — then turns that prediction into a simple score, like a credit score, that a bank employee can actually use to make a decision.

Think of it like a judge at a talent show who doesn't just say "yes" or "no" — they give every contestant a score out of 100, based on specific things they can point to (voice, stage presence, song choice). Anyone watching can see exactly why someone scored well or poorly. This project builds that same kind of transparent scoring system, but for loan risk instead of talent.

Why does a bank need this?

Every day, people apply for personal loans. The bank has to decide: approve, decline, or ask a human to look closer. Doing this by gut feeling doesn't scale to thousands of applications, and it's inconsistent — two different staff might decide differently on the same application. A **scorecard** solves this: it's a consistent, explainable, repeatable way to turn someone's financial details into a decision.

Where does the data come from?

Real bank data is private and sensitive, so this project uses **realistic fake (synthetic) data** — 9,000 made-up loan applications with made-up (but statistically realistic) customers, credit bureau records, and repayment histories. The fake data was deliberately built so that riskier-looking applicants (low income, unstable jobs, past missed payments) really do default more often — otherwise there'd be nothing real to learn from.

The four big questions this project answers

1. Who actually failed to repay their loan? ("the target")

Before you can predict something, you have to clearly define it. This project uses the standard banking definition: someone is "Bad" if they fall **90+ days behind on payments**. Everyone else is "Good." This sounds simple, but there's a subtlety: you have to wait long enough after someone takes out a loan before you can fairly judge them Good or Bad — if you check too early, you'll wrongly label people "Good" who just hadn't had time to fail yet.

A real finding from this project: we first assumed 12 months was long enough to judge someone fairly. Testing this against the data showed that assumption was wrong — about 1 in 5 people who eventually defaulted hadn't done so yet at the 12-month mark. We extended the waiting period to 18 months, which captured the real picture much better. This is explained more in Part 1's "vintage analysis" section below.

2. Which pieces of information actually predict risk?

Not everything about a person predicts loan risk equally well. This project systematically tests each piece of information (income, credit bureau score, employment type, existing debts, etc.) and measures how strongly it's connected to actually defaulting. Weak or irrelevant information gets dropped; strong information gets kept.

3. How do you combine everything into one number?

Once you know which pieces of information matter, you need a mathematical model to combine them into a single risk prediction. This project uses a well-established statistical method called **logistic regression** — it's the industry-standard choice for this exact job, because its output can be explained in plain terms ("your score dropped because of X, Y, Z") rather than being a mysterious black box.

4. How do you turn a probability into something a human can use?

A raw probability like "4.2% chance of default" isn't very actionable on its own. So the project converts that probability into a **points-based score** (similar in spirit to how your personal credit score works) — a number roughly between 500 and 650 in this project, where higher is always safer. Applicants are then grouped into five bands: Decline, Refer for Manual Review, Standard, Preferred, and Premium.

What does the final output actually look like?

At the end, every one of the 5,578 loan applications in the dataset has:

Column	Plain-English meaning
Application ID	Which application this is
Predicted PD	The model's estimated chance this person defaults (e.g. 3.5%)
Credit Score	That probability turned into a single score, like 590
Risk Band	Which of the 5 groups they fall into (Decline → Premium)
Decision	What the bank should do: Decline, Refer, or Approve (at 3 pricing tiers)

Is this scorecard actually good?

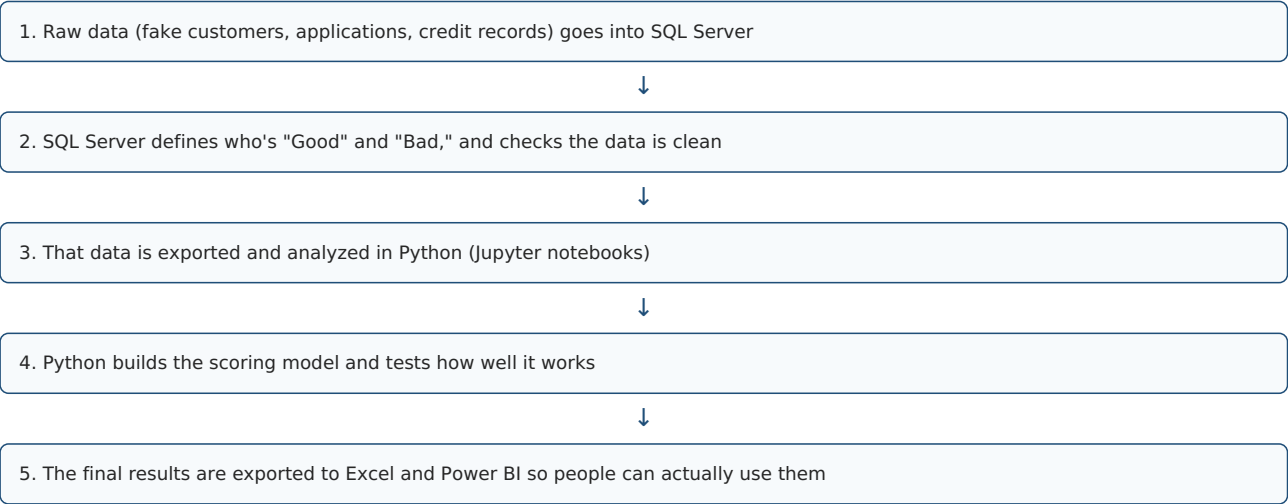
Yes, but modestly — and the project is upfront about that rather than overselling it. On a scale banks commonly use, this scorecard lands at the lower end of "acceptable," not "excellent." That's an honest, realistic outcome for a project built on a modest amount of data with a relatively low default rate (about 3.6%) — there simply weren't many actual failed loans to learn from. The project explains this clearly in its final performance report rather than hiding it.

Why the honesty matters: during development, several bugs were found where the numbers LOOKED fine but were actually backwards or wrong — for example, at one point the "credit score" was accidentally built so that riskier people got HIGHER scores, which is exactly backwards. These bugs were caught by deliberately checking "does this actually make business sense," not just "did the code run without an error." Every one of these bugs, and the fix, is documented honestly in the project's [CHANGELOG.md](#) file rather than being hidden.

Part 2 — The Tools Used, In Plain Terms

Tool	What it's for here	Analogy
SQL Server	Stores the raw data in organized tables and answers structured questions about it (e.g. "how many applications defaulted?")	A very large, very organized filing cabinet you can ask questions of
Python / Jupyter Notebooks	Does the actual analysis, math, and modeling — cleaning data, building the scorecard, testing how well it works	The workshop where the real building happens
Excel	A simple, familiar way for a non-technical person to view the final results and explore them with formulas	The report you'd actually hand to your manager
Power BI	An interactive visual dashboard for exploring the portfolio of applications, risk bands, and trends	A live, clickable version of the report, for exploring rather than just reading

The overall flow, visually



Part 3 — Step-by-Step: How to Run Every File

In the exact order to run them, with what each one does and produces

Two ways to do this: you can run every step by hand, one at a time, to see each result yourself (recommended the first time, so you actually learn what's happening) — or once you're comfortable, run `src/run_pipeline.py` to do Steps 8-14 automatically in one go. Both are explained below.

Before you start: install what you need

0 Install Python packages `requirements.txt`

Open a terminal in the project folder and run:

```
pip install -r requirements.txt
```

This installs pandas, numpy, matplotlib, scikit-learn, statsmodels, openpyxl, and everything else the project needs. You'll also need SQL Server (with SSMS) installed separately for the SQL steps, and Power BI Desktop if you want to build the dashboard.

Stage A — Get the data into SQL Server

1 Look at the raw data files `01_data/raw_csv/`

Five spreadsheet-style files already exist here: customer details, loan applications, credit bureau data, bank transactions, and monthly repayment history. These were generated for you — you don't need to create them yourself unless you want to regenerate fresh ones with `src/generate_data.py`.

2 Create the database tables `02_sql/01_create_tables.sql`

Open this file in SQL Server Management Studio (SSMS) and run it. It creates an empty database called `MTB_ScorecardProject` with five tables, ready to receive the data.

3 Load the CSV files into those tables `02_sql/02_bulk_insert_csvs.sql`

Before running this one, open it and update the file paths near the top to point to wherever you saved the CSV files on your computer. Then run it — it copies all the data from the CSV files into the SQL tables. At the bottom it prints row counts so you can confirm everything loaded correctly (expect about 9,000 applications).

4 Check the data is clean `02_sql/03_data_quality_checks.sql`

Run this to check for missing values, duplicate records, and impossible values (like a negative age). Everything should come back clean — if something looks wrong here, don't move forward until it's fixed.

5 Decide who's "Good" and who's "Bad" `02_sql/04_merge_and_target.sql`

This is where the project defines the thing it's trying to predict: did this person fall 90+ days behind on payments within 18 months of taking out the loan? Run this script — it creates the labeled dataset that everything downstream depends on.

6 Double-check the 18-month waiting period was the right call `02_sql/05_vintage_analysis.sql`

Run this to see the evidence behind the 18-month decision from Step 5. It's safe to skip running this yourself if you just want to move forward — but it's worth reading, since it shows genuine detective work: a 12-month wait was tested first and found to be too short.

At this point, all your data is organized and labeled inside SQL Server. Everything from here happens in Python.

Stage B — Explore and understand the data

7

Open and run the first notebook `03_notebooks/01_eda_cleaning.ipynb`

Open this in Jupyter and run every cell from top to bottom (in Jupyter: **Cell** → **Run All**, or click each cell and press Shift+Enter). This notebook looks at the data with charts and summary statistics — how income is distributed, which factors seem linked to default, whether anything looks broken. You'll see several charts appear. This step produces no new files — it's purely for understanding what you're working with, the same way you'd skim a report before acting on it.

Stage C — Build the scoring model

8

Turn raw numbers into risk buckets `03_notebooks/02_woe_binning.ipynb`

Run all cells. This groups each piece of information (like income) into a handful of risk-sensible buckets (e.g. "under \$30k," "\$30k-\$80k," "over \$80k") and calculates how risky each bucket actually is. **Produces:** `01_data/final_coarse_bins.json` — the bucket definitions every later step relies on.

9

Measure which factors actually matter `03_notebooks/03_information_value.ipynb`

Run all cells. This scores every candidate factor (income, credit score, employment type, etc.) on how strongly it predicts default, and builds a shortlist of the useful ones. **Produces:** a shortlist file and a lookup table used in the next step.

10

Build the actual prediction model `03_notebooks/04_logistic_regression.ipynb`

Run all cells. This splits the data into a "training" portion (used to build the model) and a "test" portion (kept aside to honestly check the model later), then fits the statistical model. It ends up keeping 4 factors: credit bureau score, prior defaults, existing debt, and employment type. **Produces:** the saved, ready-to-use model.

11

Convert the model into a points-based scorecard `03_notebooks/05_scorecard_build.ipynb`

Run all cells. This is where the statistical model becomes a scorecard a human could actually read and apply — a points table, plus 5 risk bands (Decline through Premium). **Produces:** `01_data/stage8_scorecard.json`, the final scorecard definition, plus scored versions of the training and test data.

Stage D — Check the scorecard actually works

12

Test how good the scorecard really is `03_notebooks/06_validation.ipynb`

Run all cells. This is the honesty check — it measures, using ONLY the test data the model never saw while being built, how well the scorecard actually separates risky from safe applicants. You'll see charts and a final report card of scores (this project lands at a modest, honest "acceptable" level, not an inflated "excellent" one).

Stage E — Produce the final, usable output

13

Build the final application-by-application results `03_notebooks/07_business_output.ipynb`

Run all cells. This combines everything into one final table: every application, its predicted risk, its score, its band, and the recommended decision. **Produces:** `06_deliverables/final_scored_applications.csv` — the single most important output file in the whole project.

Shortcut for Steps 7-13: once you've run through them once by hand and understand what each one does, you can run all seven automatically with a single command from the project's main folder:

```
python3 src/run_pipeline.py
```

This runs every notebook in the correct order and writes a log file to the `logs/` folder recording what happened and how long it took.

Stage F — Check nothing broke (optional but recommended)

14 Run the automatic checks `tests/test_pipeline_outputs.py`

From the project's main folder, run:

```
python3 -m pytest tests/ -v
```

This automatically checks 8 things that must be true for the output to be trustworthy — for example, that higher scores always mean lower risk, and that there are no duplicate or missing entries. You should see `8 passed`. These are the exact same checks that caught real bugs during this project's development.

Stage G — View the results the easy way

15 Open the Excel summary `04_excel/scorecard_summary.xlsx`

No running required — just open it. It contains an overview, the full list of scored applications, a live summary table by risk band, the scorecard's points table, and the performance metrics — all in one familiar spreadsheet.

16 Build the interactive dashboard (optional) `05_powerbi/powerbi_data_model_guide.md`

This is a written guide, not a ready-made file — it walks you through connecting Power BI to `final_scored_applications.csv` and SQL Server, and building the recommended charts and pages. Follow it in Power BI Desktop if you want an interactive dashboard.

Quick reference: the whole sequence at a glance

#	File	What happens
1	<code>01_create_tables.sql</code>	Create empty database tables
2	<code>02_bulk_insert_csvs.sql</code>	Load the raw data in
3	<code>03_data_quality_checks.sql</code>	Confirm the data is clean
4	<code>04_merge_and_target.sql</code>	Define Good vs Bad customers
5	<code>05_vintage_analysis.sql</code>	Confirm the 18-month rule is correct
6	<code>01_eda_cleaning.ipynb</code>	Explore and understand the data
7	<code>02_woe_binning.ipynb</code>	Group data into risk buckets
8	<code>03_information_value.ipynb</code>	Find which factors matter most
9	<code>04_logistic_regression.ipynb</code>	Build the prediction model
10	<code>05_scorecard_build.ipynb</code>	Turn it into a points-based scorecard
11	<code>06_validation.ipynb</code>	Test how good it really is
12	<code>07_business_output.ipynb</code>	Produce the final results file
13	<code>pytest tests/</code>	Automatically confirm nothing is broken
14	<code>scorecard_summary.xlsx</code>	View results in Excel
15	Power BI guide	Build an interactive dashboard (optional)

If something goes wrong: check `logs/` for the most recent pipeline run log — it records exactly which step failed and why. `CHANGELOG.md` also documents every real bug found during this project's own development, in case the issue you're hitting looks familiar.

Where to go next: `README.md` has the full technical setup instructions. `00_README.md` explains the credit-risk reasoning behind every stage in much more depth than this guide. `CHANGELOG.md` is genuinely worth reading — it tells the honest story of every mistake made and fixed while building this, which is often the most instructive part of a project like this.